Senior Full Stack Developer
Java + React Interview Prep

2-hour focused revision pack | 10–12 years experience | Java, React, Microservices, AWS & GenAI

How to use this guide

Prioritize clear explanations over memorizing. For every scenario, explain: problem → approach → implementation → scalability → failure handling → monitoring.

Suggested 120-minute plan

Time	Focus	What to revise
0–20 min	Introduction + Java	Self-introduction, overloading, overriding, String Pool
20–45 min	Java coding + concurrency	LRU cache, HashMap, locks, thread safety
45–75 min	React + full stack	Architecture, API integration, WebSockets, performance
75–100 min	Microservices + production	Retries, circuit breakers, messaging, incidents
100–112 min	AWS + GenAI	RAG, agents, Bedrock, SQS, Lambda
112–120 min	Final revision	Speak answers aloud; review checklist

Important: Use examples from your real projects. Do not claim hands-on experience with tools you have only studied.

1. Self-introduction

Question: Tell me about yourself, your current project, role and responsibilities.

Speaking template (customize with your actual details):

"I am a Senior Full Stack Developer with around 10 years of experience in software development, primarily working with frontend technologies such as React, Angular, JavaScript and TypeScript, along with backend integration and enterprise application development. In my current project, I work on end-to-end features, reusable UI components, REST API integration, authentication, application performance and production support. I participate in requirement discussions, technical design, code reviews, debugging and coordination with cross-functional teams. My focus is building maintainable, scalable and reliable applications while ensuring a good user experience."

Likely follow-ups: What is your architecture? What exactly do you own? What was your biggest technical challenge? How do you improve performance? How do you troubleshoot production issues?

For project architecture, be ready to describe: frontend → API gateway/load balancer → backend services → database/cache/message broker → monitoring/deployment.

2. Java fundamentals

Method overloading vs overriding

Overloading: Same method name, different parameter list. Usually compile-time polymorphism.

```
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}
```

Overriding: A child class provides a compatible implementation of an inherited parent method. Runtime polymorphism.

```
class Animal {
    void sound() { System.out.println("Animal sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Bark"); }
}
Animal a = new Dog();
a.sound(); // Bark
```

Can an overriding method change its return type?

Yes, for reference types, Java allows a **covariant return type**: the child method may return a subtype of the parent's declared return type. Primitive return types must match exactly.

```
class Animal { Animal get() { return new Animal(); } }
class Dog extends Animal {
    @Override Dog get() { return new Dog(); }
}
```

If the parent returns **char** and the child returns **int**, overriding is not allowed. A common reference return type can make a related design possible:

```
class Parent { Object getValue() { return 'A'; } }
class Child extends Parent {
    @Override Integer getValue() { return 10; }
}
```

Remember: return type alone cannot distinguish overloaded methods.

String Pool

The String Pool is a JVM-managed pool of canonical String instances used to reuse equal string literals. String objects are immutable.

```
String s1 = "hello";
String s2 = "hello";
System.out.println(s1 == s2);      // true
System.out.println(s1.equals(s2)); // true
```

Memory for: String s = new String("hello")

- The literal "hello" is represented in the String Pool (created there if not already present).
- new String("hello") creates a separate String object on the heap.
- If s is a local variable, the reference variable is in the current stack frame and points to the new heap object.

```
String s1 = "hello";
String s2 = new String("hello");
System.out.println(s1 == s2);      // false
System.out.println(s1.equals(s2)); // true
String s3 = s2.intern();
System.out.println(s1 == s3);      // true
```

Interview nuance: Avoid saying the literal is always newly created. It may already exist in the pool. The exact object layout is JVM-implementation dependent; explain the logical memory model.

3. LRU Cache – coding round

Requirement: Fixed capacity; O(1) average get and put; implement manually; thread-safe.

Design: HashMap maps key → node for O(1) average lookup. A doubly linked list tracks recency. Head = most recently used; node before tail = least recently used. On get, move node to front. On put, update/move existing node or insert new node; if full, evict tail.prev.

Example, capacity 3: put(1), put(2), put(3) → order 3→2→1. get(1) → 1→3→2. put(4) evicts key 2.

```
import java.util.*;
import java.util.concurrent.locks.ReentrantLock;

class LRUCache {
    class Node {
        int key, value;
        Node prev, next;
        Node(int k, int v) { key = k; value = v; }
    }

    private final int capacity;
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(0, 0);
    private final Node tail = new Node(0, 0);
    private final ReentrantLock lock = new ReentrantLock();

    LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    private void remove(Node n) {
        n.prev.next = n.next;
        n.next.prev = n.prev;
    }

    private void addFront(Node n) {
        n.next = head.next;
        n.prev = head;
        head.next.prev = n;
        head.next = n;
    }

    public int get(int key) {
        lock.lock();
        try {
            Node n = map.get(key);
            if (n == null) return -1;
            remove(n);
            addFront(n);
            return n.value;
        } finally { lock.unlock(); }
    }

    public void put(int key, int value) {
        if (capacity <= 0) return;
        lock.lock();
        try {
            Node n = map.get(key);
            if (n != null) {
                n.value = value;
                remove(n);
                addFront(n);
                return;
            }
            if (map.size() == capacity) {
                Node lru = tail.prev;
                remove(lru);
                map.remove(lru.key);
            }
            n = new Node(key, value);
            map.put(key, n);
            addFront(n);
        } finally { lock.unlock(); }
    }
}
```

Explain: HashMap gives constant-time average lookup; doubly linked list gives constant-time removal and movement. A single ReentrantLock protects both structures as one consistent operation.

ReentrantLock vs ReadWriteLock

A strict LRU get is not a pure read: it changes recency order by moving the node. Therefore, both get and put need the write lock if using ReadWriteLock. A read lock for get is unsafe if it modifies the list.

To allow concurrent reads, redesign recency tracking (for example, approximate/batched updates or separate access metadata). This adds complexity and may weaken strict LRU ordering.

4. React + full-stack integration

How do you integrate React with Java/Spring Boot?

React is the presentation layer and Spring Boot exposes backend APIs. React calls REST endpoints using Fetch/Axios. Backend services handle business logic, validation, authentication/authorization and persistence. Keep API contracts clear, handle loading/errors, and deploy frontend/backend independently where appropriate.

Typical architecture: React + TypeScript → API Gateway / Load Balancer → Spring Boot services → database/cache/message broker.

Integration challenge example

If it matches your experience: a dashboard was slow because several API calls loaded sequentially. Use browser Network tools to identify bottlenecks, parallelize independent requests, add pagination/caching, and handle partial failures.

```
const results = await Promise.all([
  getUsers(), getOrders(), getProducts()
]);
// Use Promise.allSettled() if each result
// should be handled even when another fails.
```

Authentication and API errors

- For token-based auth, send access tokens securely and validate signature, issuer, expiry and permissions on the backend. Never put secrets or private API keys in React.
- For cookie sessions, use Secure, HttpOnly and appropriate SameSite settings, plus CSRF protection where needed.
- Handle 401 (authentication), 403 (authorization), 429 (rate limit), and 5xx (server errors) appropriately.

REST vs WebSocket

REST is request/response and suits CRUD. WebSocket is a persistent, bidirectional connection and suits live notifications, chat and dashboards.

10,000 live devices scenario: authenticate connections; scale connection handling; distribute events through a broker when needed; virtualize the UI; batch updates; use event IDs/sequence numbers; ignore duplicates/stale events; reconnect and resync state through REST.

Out-of-order events: use sequence/version numbers. **Duplicate events:** idempotent processing and event IDs.

5. React performance

Start by measuring with React DevTools Profiler, browser Performance tools and real-user metrics. Optimize the actual bottleneck.

Technique	Purpose
React.memo	Skip re-render when props are unchanged (subject to comparison)
useMemo	Memoize an expensive computed value
useCallback	Memoize a function reference
Lazy loading/code splitting	Defer code until needed
Virtualization	Render only visible rows/items

Technique	Purpose
Debouncing	Reduce repeated actions/API calls
Caching	Reuse server data; e.g. React Query/TanStack Query

Lazy loading and measurement

```
const Dashboard = React.lazy(() => import("./Dashboard"));

function App() {
  return <React.Suspense fallback={<div>Loading...</div>}>
    <Dashboard />
  </React.Suspense>;
}
```

Measure initial bundle size, FCP, LCP, route-level load time, Lighthouse and real-user monitoring. Lazy loading reduces initial code but can add a loading delay when the deferred component is first opened.

Context vs Redux

Context is built into React and works for relatively simple shared state. Redux offers a more structured state-management model and tooling for complex application state. Choose based on complexity, update patterns and team needs.

6. Microservices and production

Scalable microservice design

Split services around business domains, not arbitrary technical layers. Keep services stateless where possible, give each service clear ownership of its data, scale instances horizontally, and use synchronous REST/gRPC or asynchronous messaging based on the use case.

Timeout, retry, exponential backoff, jitter, circuit breaker

Pattern	Meaning
Timeout	Stop waiting after a configured duration
Retry	Try again for transient failures, with limits
Exponential backoff	Increase delay, e.g. 1s, 2s, 4s
Jitter	Add randomness to avoid synchronized retries
Circuit breaker	Temporarily stop calls to an unhealthy dependency

Circuit breaker states: **Closed** = calls flow; **Open** = calls blocked; **Half-open** = limited test calls check recovery.

Do not retry blindly: validation/authentication errors and non-idempotent payment writes. Respect Retry-After, cap retries, use timeouts, and use idempotency keys for safe write retries.

Service discovery, load balancing, Kafka vs SQS

- Service discovery lets services find current service instances (e.g. AWS Cloud Map or Kubernetes DNS).
- Load balancing distributes requests across healthy instances (e.g. AWS Application Load Balancer).
- Kafka is a distributed event-streaming platform with retained events and replay/consumer groups.
- SQS is a managed queue for decoupled asynchronous message processing.

High availability and production incident

Use multiple instances across failure domains, health checks, load balancing, backups, observability, alerting, deployment safeguards and tested recovery procedures.

Incident story – use a real example: Situation (what broke) → Task (your responsibility) → Action (logs, metrics, traces, isolate dependency/database, mitigate, fix) → Result (downtime/customer impact/latency reduced).

Tools to recognize: CloudWatch (AWS metrics/logs/alarms), Prometheus (metrics), Grafana (dashboards), ELK/OpenSearch (logs), OpenTelemetry/Jaeger (tracing).

7. GenAI and AWS – JD-specific essentials

What is RAG?

Retrieval-Augmented Generation grounds an LLM response in retrieved external information.

- Ingest documents and split them into chunks.
- Create embeddings and store them in a vector database.
- Embed the user's question and retrieve relevant chunks.
- Send the question plus retrieved context to the LLM.
- Generate an answer, ideally with source references and safeguards.

Embedding: numerical vector representing semantic information. **Vector database:** stores/searches vectors by similarity. Examples include FAISS, Pinecone and OpenSearch vector search.

What is an AI agent?

An LLM-powered system that can choose actions, use tools and carry out multi-step tasks toward a goal. A workflow follows predefined steps; an agent can dynamically decide the next step. Agents need limits, permission checks, validation and observability.

How to integrate GenAI into React + Java

Keep model credentials and orchestration on the backend. React calls an authenticated API; Spring Boot validates the user and request, then calls an AI service (possibly Python/FastAPI). The AI service can retrieve context from a vector store and call an LLM. Add rate limits, timeouts, token/cost tracking, audit logs, prompt-injection defenses and output validation.

AWS services – one-line revision

Service	Purpose
EC2	Virtual machines/servers
ECS	Managed container orchestration
Fargate	Run containers without managing servers
Lambda	Serverless event-driven functions
S3	Object storage
DynamoDB	Managed NoSQL database
SQS	Managed message queue
Step Functions	Coordinate multi-step workflows
Bedrock	Managed access to foundation models and GenAI tools

8. Additional senior-level quick questions

How do you design scalable REST APIs? Pagination, filtering, validation, versioning, caching, clear contracts and observability.

PUT vs PATCH? PUT replaces the resource representation; PATCH applies a partial update.

How do you prevent duplicate transactions? Idempotency keys, unique constraints and transactional safeguards.

How do you handle database concurrency? Transactions, optimistic locking or pessimistic locking based on contention and requirements.

How do you handle microservice data inconsistency? Eventual consistency, Saga pattern and transactional outbox where appropriate.

JWT? A signed token with header, payload and signature; validate signature and relevant claims.

Authentication vs authorization? Authentication establishes identity; authorization determines permitted actions.

Dependency injection in Spring? The container supplies dependencies to components, reducing tight coupling.

Synchronous vs asynchronous communication? Synchronous callers wait for a response; asynchronous work is decoupled through messaging/events.

How do you process large Python datasets? Chunking, generators, vectorized operations, async I/O for I/O-bound work and multiprocessing/distributed tools when appropriate.

9. Final 20-question revision checklist

1. [ ] Overloading vs overriding
2. [ ] Covariant return type
3. [ ] String Pool and intern()
4. [ ] Stack vs Heap
5. [ ] HashMap internal working
6. [ ] HashMap vs ConcurrentHashMap
7. [ ] LRU Cache with HashMap + doubly linked list
8. [ ] ReentrantLock vs ReadWriteLock
9. [ ] Thread safety and race conditions
10. [ ] React.memo vs useMemo vs useCallback
11. [ ] React rendering and reconciliation
12. [ ] Virtual DOM
13. [ ] React lazy loading
14. [ ] REST vs WebSocket
15. [ ] JWT authentication and authorization
16. [ ] Microservices communication
17. [ ] Timeout vs Retry vs Circuit Breaker
18. [ ] Kafka vs SQS
19. [ ] RAG vs AI Agent
20. [ ] AWS Lambda vs EC2 vs ECS

10. Answer like a senior engineer

For scenario questions, use this structure:

- Problem: What are you solving?
- Approach: Which architecture or technology and why?

- Implementation: How would you build it?
- Scalability: What changes when traffic increases?
- Failure handling: What if a dependency or node fails?
- Monitoring: Which metrics, logs, traces and alerts prove it works?

Example: thousands of concurrent requests

“I would use stateless horizontally scalable services behind a load balancer, connection pooling, asynchronous processing for background work, caching and database optimization. I would also configure rate limiting, timeouts, circuit breakers and monitoring to protect the system under load.”

Final reminder: Use truthful examples from your own work. If you have not implemented a tool hands-on, explain the concept and how you would approach it rather than claiming experience.